

Algorithms

Lecture #42

Syed Hamed Raza

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

- The algorithm dates back to the early 60's.
- As with other dynamic programming algorithms, the genius of the algorithm is in the clever recursive formulation of the shortest path problem.
- For a path $p = \langle v_1, v_2, \dots, v_l \rangle$, we say that the vertices $v_2, v_3, \dots, v_{l-1}$ are the **intermediate vertices** of this path.

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

Formulation:

- Define $d_{ij}^{(k)}$ to be the shortest path from i to j such that any intermediate vertices on the path are chosen from the set $\{1, 2, \dots, k\}$.
- The path is free to visit any subset of these vertices and in any order.

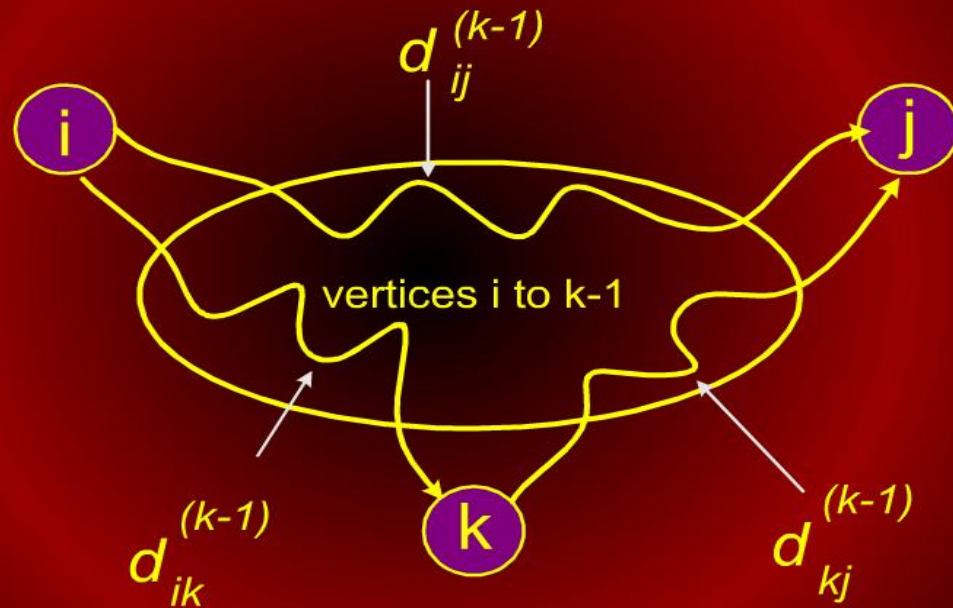
Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

- How do we compute $d_{ij}^{(k)}$ assuming we already have the previous matrix $d^{(k-1)}$?
- There are two basic cases:
 1. Don't go through k at all
 2. Do go through k

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm



Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

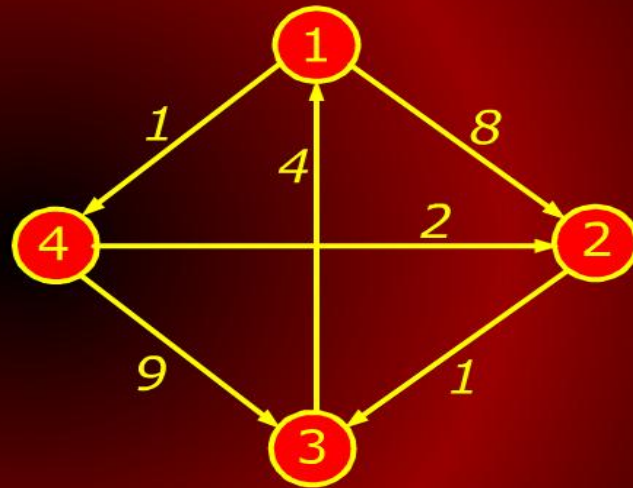
1. Don't go through k at all
 - Then the shortest path from i to j uses only intermediate vertices $\{1, 2, \dots, k-1\}$. Hence the length of the shortest is $d_{ij}^{(k-1)}$.

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

0	8	∞	1
∞	0	1	∞
4	∞	0	∞
∞	2	9	0

$d^{(0)}$



Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

2. Do go through k

- First observe that a shortest path does not go through the same vertex twice, so we can assume that we pass through k exactly once. That is, we go from i to k and then from k to j .
- In order for the overall path to be as short as possible, we should take the shortest path from i to k and the shortest path from k to j .

Floyd-Warshall Algorithm

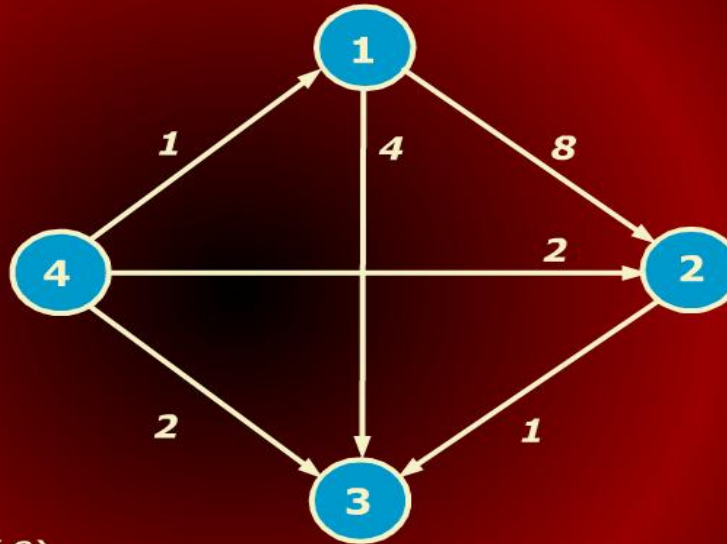
Floyd-Warshall Algorithm

2. Do go through k

- Since each of these paths uses intermediate vertices $\{1, 2, \dots, k-1\}$, the length of the path is $d_{ij}^{(k-1)} + d_{kj}^{(k-1)}$

Floyd-Warshall Algorithm

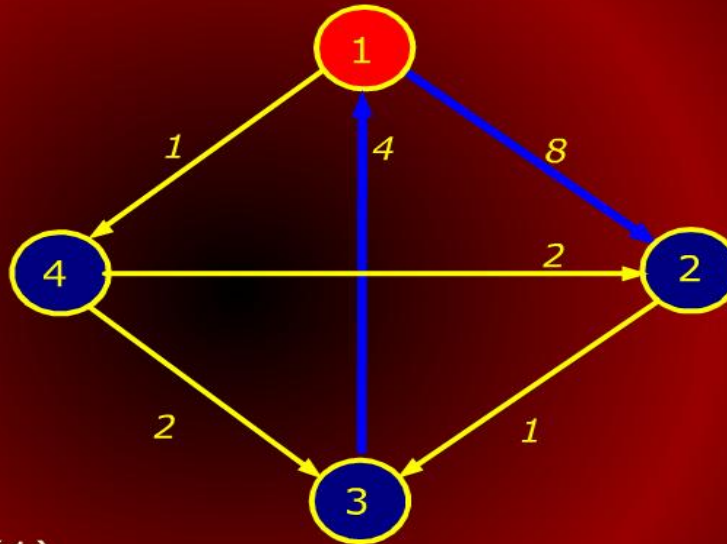
Floyd-Warshall Algorithm



$k = 0, d_{3,2}^{(0)} = \infty$ (no path)

Floyd-Warshall Algorithm

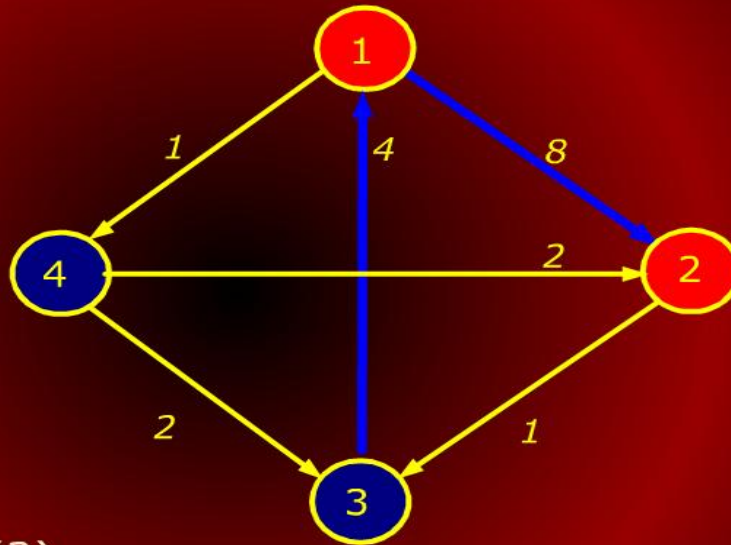
Floyd-Warshall Algorithm



$k = 1, d_{3,2}^{(1)} = 12 \text{ (} 3 \rightarrow 1 \rightarrow 2 \text{)}$

Floyd-Warshall Algorithm

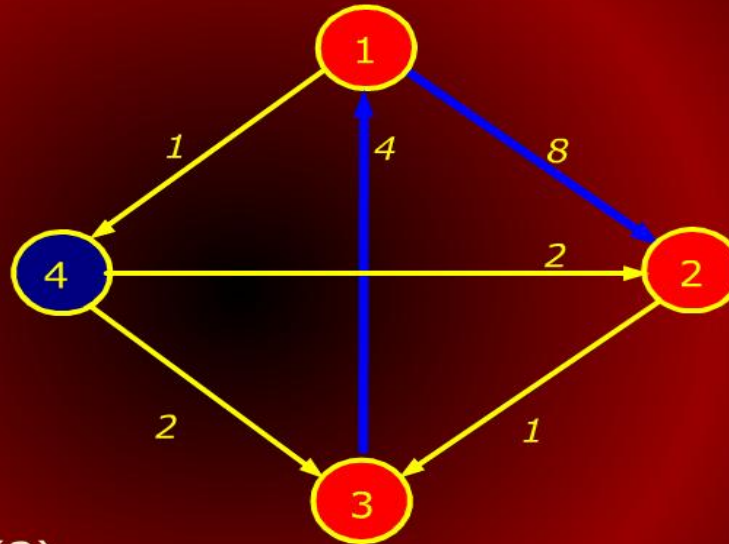
Floyd-Warshall Algorithm



$$k = 2, d_{3,2}^{(2)} = 12 \quad (3 \rightarrow 1 \rightarrow 2)$$

Floyd-Warshall Algorithm

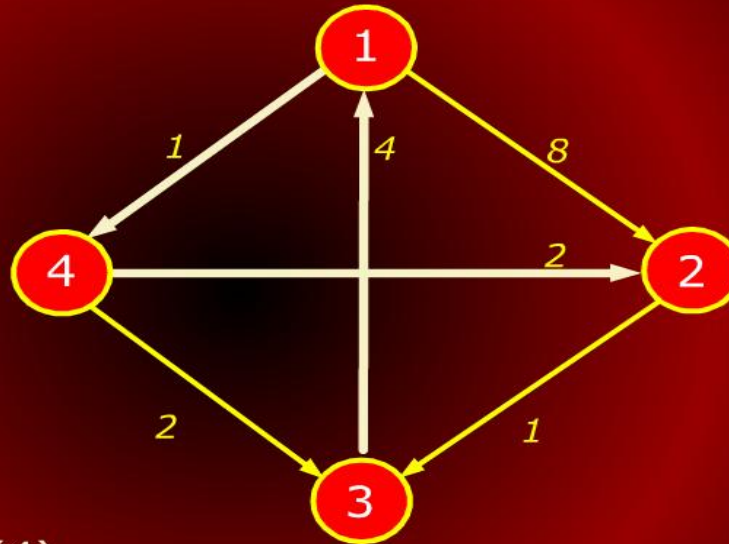
Floyd-Warshall Algorithm



$$k = 3, d_{3,2}^{(3)} = 12 \quad (3 \rightarrow 1 \rightarrow 2)$$

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm



$k = 4, d_{3,2}^{(4)} = 7 \text{ (} 3 \rightarrow 1 \rightarrow 4 \rightarrow 2 \text{)}$

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

- This suggests the following recursive (DP) formulation:

$$d_{ij}^{(0)} = w_{ij}$$

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)}, d_{kj}^{(k-1)})$$

- The final answer is $d_{ij}^{(n)}$ because this allows all possible vertices as intermediate vertices.

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

- As is the case with DP algorithms, we will avoid recursive evaluation by generating a table for $d_{ij}^{(k)}$.
- The algorithm also includes mid-vertex pointers stored in $mid[i, j]$ for extracting the final path.

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

FLOYD-WARSHALL($n, w[1..n, 1..n]$)

1 **for** ($i = 1, n$)

2 **do for** ($j = 1, n$)

3 **do** $d[i, j] \leftarrow w[i, j]; \text{mid}[i, j] \leftarrow \text{null}$

4 **for** ($k = 1, n$)

5 **do for** ($i = 1, n$)

6 **do for** ($j = 1, n$)

7 **do if** ($d[i, k] + d[k, j] < d[i, j]$)

8 **then** $d[i, j] = d[i, k] + d[k, j]$

9 $\text{mid}[i, j] = k$

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

- Clearly, the running time is $\Theta(n^3)$.
- The space used by the algorithm is $\Theta(n^2)$.

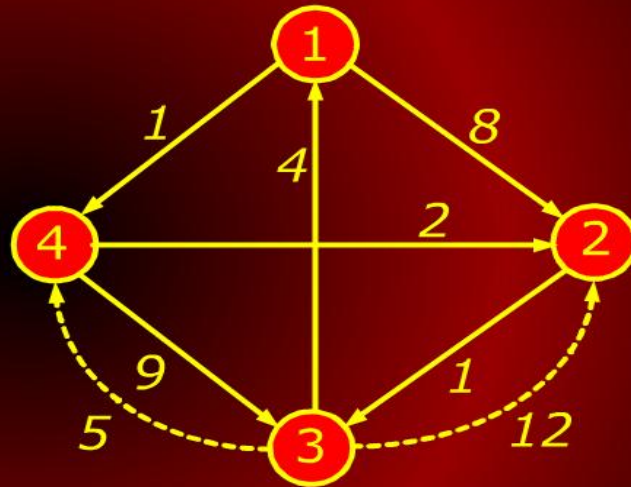
Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

0	8	∞	1
∞	0	1	∞
4	12	0	5
∞	2	9	0

$d^{(1)}$

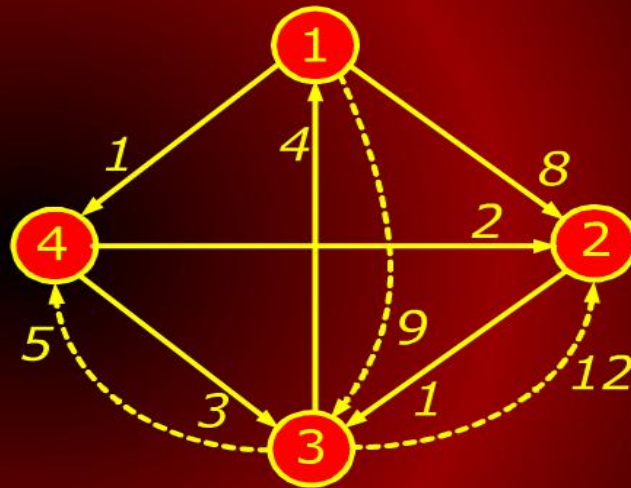


Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

0	8	9	1
∞	0	1	∞
4	12	0	5
∞	2	3	0

$d^{(2)}$

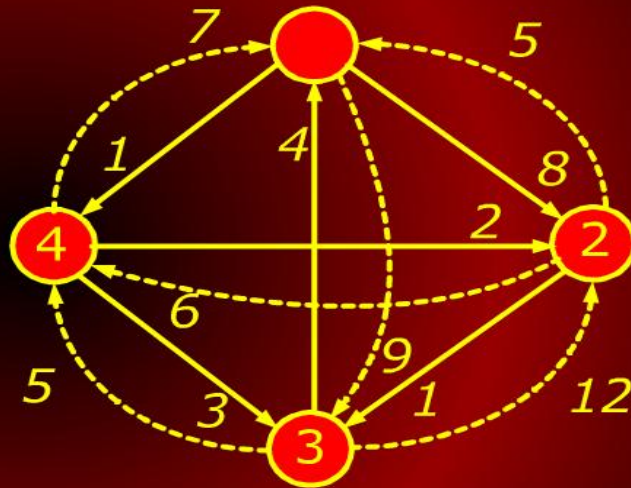


Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

0	8	9	1
5	0	1	6
4	12	0	5
7	2	3	0

$d^{(3)}$

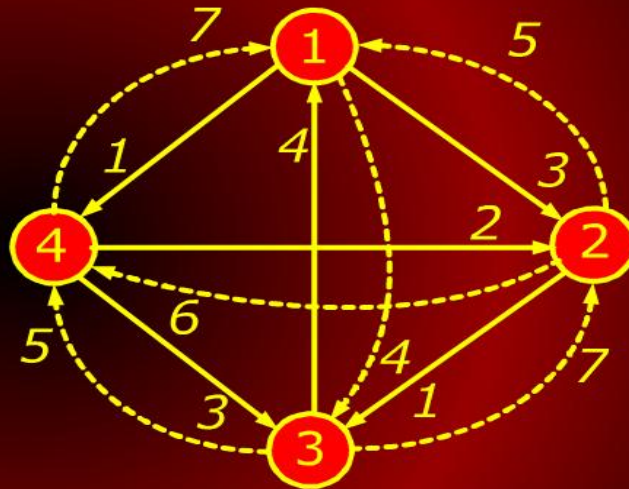


Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

0	3	4	1
5	0	1	6
4	7	0	5
7	2	3	0

$d^{(4)}$



Criteria for analyzing Algorithms

Model of Computation